

Advanced Computing Project (2025/26)

Storage system for optimizing LLM checkpointing in HPC

André Miranda
MEI, PG60238

Gustavo Barros
MEI, PG61527

José Soares
MEI, PG60277

Miguel Carvalho
MEI, PG61153

Abstract—Training Large Language Models (LLMs) in High-Performance Computing (HPC) environments is a time-consuming process that requires frequent checkpointing to ensure fault tolerance. However, concurrent writes of massive model states often saturate the shared Parallel File System (PFS), leading to significant I/O contention and training stalls. This paper proposes a policy-driven, two-tier storage orchestration system designed to mitigate the “noisy neighbor” problem in HPC clusters. By transparently redirecting checkpoints to node-local SSDs and employing a centralized orchestrator to manage asynchronous migration to the PFS via a Token Bucket algorithm, we enable granular Quality of Service (QoS) control. Our evaluation on the Deucalion supercomputer demonstrates that

...
Index Terms—High-Performance Computing, Deucalion, A64FX, AMD EPYC 7742, NVIDIA A100, Memory Hierarchy.

I. INTRODUCTION

TODO

II. ...

TODO

III. BENCHMARKING

TODO

IV. RELATED WORK

The optimization of checkpointing in High-Performance Computing (HPC) has been extensively studied, primarily focusing on reducing write latency and mitigating Parallel File System (PFS) congestion.

A. Architectural and Tiering Approaches

Several systems utilize hierarchical storage to hide I/O latency. **Check-N-Run** focuses on staging checkpoints in local node resources (DRAM or SSDs), allowing training to resume immediately while data is moved to the PFS in the background. However, it lacks a global coordination mechanism to prevent PFS saturation during the background flush. **DeepFreeze** (similar to Monarch) employs transparent storage tiering to accelerate the write path by abstracting complex local tiers like NVMe or PMEM from the user. While effective at the node level, these systems often ignore the “noisy neighbor” effect in shared environments.

B. Adaptive and Optimization Techniques

Other research focuses on the frequency and size of checkpoints. **CheckFreq** introduces an algorithmic approach to identify optimal checkpoint boundaries and pipelines the process with training iterations, dynamically adjusting frequency to maintain overhead below a threshold. **BitSnap** addresses the data volume by employing sparsification and quantization, reducing checkpoint sizes from 16-bit to 4-bit weights before they reach the I/O layer. While these reduce total I/O pressure, they do not manage the storage hierarchy or provide holistic Quality of Service (QoS) guarantees.

C. Software-Defined Storage and QoS Control

Our work is heavily informed by **PADLL**, a storage middleware that enables application-agnostic QoS control for both data and metadata workflows. PADLL utilizes a decoupled Software-Defined Storage (SDS) architecture, where data plane stages intercept POSIX calls and a centralized control plane coordinates I/O rates across all jobs. While PADLL is a general-purpose QoS framework, our system specializes this paradigm for the specific lifecycle of LLM checkpoints—integrating node-local SSD staging with PADLL-inspired Token Bucket rate limiting to ensure cluster harmony during massive model state migrations.

V. CONCLUSIONS

TODO

REFERENCES

- [1] EuroHPC Joint Undertaking, “Access Calls – Technical Guidelines, Version 6 (26.05.2025),” 2025. Link
- [2] Fujitsu Ltd., “FUJITSU Processor A64FX – Architecture and Specifications,” 2020. Link
- [3] AMD, “AMD EPYC 7742 Processor (Rome) – Product Specifications and Performance Guide,” 2019–2024. Link